Үйлдлийн систем — Лаборатори 3

Сэдэв: Process Control Block (PCB) ба процессын симуляци

---

1. Зорилго

Энэхүү лабораторийн ажлын зорилго нь үйлдлийн системийн **процесс ба Process Control Block (PCB)**-ийн үндсэн ойлголтыг програмчлалын түвшинд загварчлан ойлгоход оршино.

Тус лабораторийн хүрээнд оюутнууд:

- Процессийн үндсэн шинжүүд болох **process ID, priority, state, process counter** зэрэг ойлголтыг өгөгдлийн бүтэц (struct) ашиглан загварчилна.
- PCB бүтэц ашиглан процессийн төлөв болон гүйцэтгэлийн мэдээллийг хадгалах, шинэчлэх механизмыг хэрэгжүүлнэ.
- Өөр өөр логик бүхий 3 “процесс”-ийг тус тусад нь функц хэлбэрээр боловсруулж, нэг програм дотор удирдан ажиллуулах замаар олон процессын концепцийг ойлгоно.
- Процессуудыг ээлжлэн (эсвэл сонгон) ажиллуулах замаар **process scheduling ойлголтыг simulation хэлбэрээр туршина.**

Энэхүү лабораторийн ажил нь:

Жинхэнэ үйлдлийн системийн kernel түвшний процесс үүсгэх (fork, exec гэх мэт) үйлдэл биш бөгөөд
харин процессын удирдлагын логик болон PCB бүтцийг ойлгуулах зорилготой **симуляци програм** юм.

Өөрөөр хэлбэл:

- Энд бодит OS процесс үүсэхгүй.
- Context switching бодитоор хийгдэхгүй.
- Scheduler нь kernel биш, харин програмын түвшинд загварчлагдсан байна.

Гэвч энэхүү симуляци нь:

- PCB ямар мэдээлэл хадгалдаг

- Процесс state хэрхэн өөрчлөгддөг
- Нэг систем дотор олон процесс хэрхэн удирдагддаг

гэдгийг ойлгоход суурь ойлголт өгнө.

🎯 Лабораторийн ажлын хүрээ

Энэхүү лабораторийн ажлын хүрээнд:

- PCB бүтэц тодорхойлно.
- 3 өөр процессын логик бичнэ.
- Процесс бүрийн гүйцэтгэлийн тоолуур (process_counter)-ийг шинэчилнэ.
- Процессийн төлөв (READY, RUNNING гэх мэт)-ийг симуляци байдлаар удирдана.
- Эдгээрийг нэг програмд нэгтгэн ажиллуулна.

Лабораторийн ажлын дараа оюутан:

- PCB бол үйлдлийн системийн процесс удирдах үндсэн өгөгдлийн бүтэц гэдгийг ойлгоно.
- Процесс бол зөвхөн код биш, харин төлөв, тоолуур, удирдлагын мэдээлэл агуулсан объект гэдгийг ойлгоно.
- Scheduling ба context switching сэдвийг судлахад бэлтгэл суурьтай болно.

2. Ашиглах хэл, орчин

- Хэл: **C**
- Компилятор: gcc
- OS: Linux / Windows (аль аль нь боломжтой)

3. Ерөнхий шаардлага (бүх процессод хамаарна)

1. **PCB бүтэц (struct PCB)**-ийг заавал зарлана.
2. Процесс бүрийн үндсэн ажиллагаа **мөнхийн давталт** (while(1) эсвэл for(;;)) дотор ажиллана.
3. Програм **CTRL + C** дарахад зогсох (терминалаас таслах) байдлаар ажиллана.
4. Процесс бүр:

    - process_id, priority, state, process_counter утгуудтай байна.
    - process_counter нь давталт бүрт өснө (эсвэл тодорхой дүрмээр шинэчлэгдэнэ).
5. random()/rand() ашиглах бол:
    - srand(time(NULL));-ийг **эхэнд нь нэг удаа** дуудах.

4. PCB бүтэц (заавал)

```
struct PCB {
   int process_id;        // Process ID
   int priority;          // Process Priority
   int state;             // Process State (ж: READY/RUNNING/WAITING)
   int process_counter;   // Process counter value
   // Add other fields as necessary
};
```

Санал болгож буй state-ийн утгууд

- 0 = READY
- 1 = RUNNING
- 2 = WAITING
- 3 = TERMINATED (заавал биш)

5. Процесс тус бүрийн даалгавар

Process 1 — Random тоонууд үүсгэж өсөхөөр эрэмбэлэх

Үүрэг

1. 0-ээс n-1 хүртэлх индекс бүхий n ширхэг тоог random-оор үүсгэнэ.
2. Үүссэн тоонуудыг **өсөхөөр** эрэмбэлнэ.
3. Үр дүнг хэвлэнэ.
4. Энэ бүх үйлдэл **мөнхийн давталт** дотор хийгдэнэ.

Оролт/параметр

- n (жишээ: 10, 20 гэх мэт) — массивын хэмжээ

Гаралт

- Эхний random массив

- Эрэмбэлсний дараах массив
- PCB мэдээлэл (process_id, state, counter гэх мэт)

Файл

- Process1.c

Process 2 — 2 хэмжээст матриц үүсгээд нэмэх (A + B)

Үүрэг

1. A[m][n] болон B[m][n] матрицуудын элементүүдийг random-оор үүсгэнэ.

   (Нэмэх үйлдэл хийхийн тулд хэмжээс адил байх шаардлагатай. Хэрэв заавал өөр нэршлээр A[m,n], B[k,l] гэж байвал **нэмэхээс өмнө хэмжээсийг тэнцүүлэх** дүрэм оруулна.)

2. C[m][n] матрицад нийлбэрийг хадгална: C[i][j] = A[i][j] + B[i][j]
3. A, B, C-г хэвлэнэ.
4. **Мөнхийн давталт** дотор ажиллана. CTRL+C-ээр таслагдана.

Оролт/параметр

- m, n — мөр, баганын хэмжээ

Гаралт

- A матриц
- B матриц
- C = A + B матриц
- PCB мэдээлэл

Файл

- Process2.c

Process 3 — Тэмдэгт мөр доторх ижил тэмдэгтийн давтамж тоолох

Үүрэг

1. Дурын тэмдэгт мөр (string) ашиглан **давхардсан/ижил тэмдэгтүүдийн тоо**-г тоолно.
    - Ж: "hello" → h:1, e:1, l:2, o:1
2. Үр дүнг хэвлэнэ.
3. **Мөнхийн давталт** дотор давтан ажиллана. CTRL+C-ээр таслагдана.

Оролт

- String (хатуу бичсэн байж болно эсвэл random үүсгэж болно)

Гаралт

- Эх string
- Тэмдэг бүрийн давтамж (ASCII 256 хүснэгтээр тоолж болно)
- PCB мэдээлэл

Файл

- Process3.c

6. Нэгтгэх даалгавар (Final program)

Дээрх 3 процессын логикийг **нэг програм** болгон нэгтгэнэ.

Нэгтгэх шаардлага (сонголтууд)

Та 2 аргын аль нэгийг сонгож болно:

А хувилбар — “Menu + нэг процесс ажиллуулах”

- Програм ажиллахад цэс гаргана:
    1. Process1
    2. Process2
    3. Process3
    4. Exit
- Сонгосон процесс мөнхийн давталтаар ажиллана (CTRL+C-ээр зогсоно)

В хувилбар — “3 процессыг ээлжлэн ажиллуулах (Round-robin simulation)”

- Нэг while(1) давталт дотор:
  - Process1-ийн нэг алхам ажиллуулна
  - Process2-ийн нэг алхам ажиллуулна
  - Process3-ийн нэг алхам ажиллуулна
- Процесс бүрийн PCB.state, process_counter-ыг шинэчилнэ
- Хугацааны “тасалдал” мэт болгохын тулд sleep(1) ашиглаж болно (заавал биш)

Эцсийн файл нэр

- Lab3.c (эсвэл main.c)

7. Хүргэлтийн бүтэц (submission)

- Process1.c
- Process2.c
- Process3.c
- Lab2.c (эсвэл main.c)
- (Хэрэв шаардвал) README.txt — хэрхэн compile/run хийх

Жишээ:

- gcc Lab3.c -o lab3
- ./lab3